

Lucene: Java Volltextsuche & mehr

<https://gemini.google.com/app/9df29b59f51bef98>

User prompt: was kann man mit dieser Bibliothek für Java Lucine alles so anstellen?

Response: Mit Apache Lucene lässt sich eine leistungsfähige **Volltextsuche** in Java-Anwendungen integrieren.

Hauptanwendungsbereiche:

- **Suchfunktionen bauen:** Schnelle Textsuche mit Tippfehlertoleranz (Fuzzy Search), Auto-Complete, Wildcards und Facetten-Filtern.
- **Daten indexieren:** Verarbeiten und Durchsuchen riesiger Textmengen (Dokumente, E-Mails, Datenbanksätze) über sogenannte *Inverted Indexes*.
- **Relevanz-Scoring:** Suchergebnisse nach Relevanz sortieren (z. B. mit Algorithmen wie BM25).
- **Vektorsuche:** Durchführen von Ähnlichkeitssuchen (Nearest-Neighbor-Search) für KI- und Embedding-Uwendungsfälle.
- **Basis für Suchserver:** Lucene dient als Kern für bekannte Enterprise-Suchmaschinen wie **Elasticsearch**, **Apache Solr** und **OpenSearch**.

User prompt: Äh angenommen, ich will irgendwie Webseiten ähm speichern, weil ich etwas recherchiere und will mir die relevanten Webseiten speichern für meine wissenschaftliche Ausarbeitung.

Response: Für diesen Anwendungsfall ist **Apache Lucene** die ideale technische Grundlage, um eine eigene **Wissensdatenbank mit Volltextsuche** aufzubauen.

Was du damit konkret umsetzen kannst:

1. Volltext-Indexierung von Webseiten

Anstatt nur URLs zu speichern, liest du den gesamten Text einer Webseite ein. Lucene bereitet diesen Text auf (*Analyse, Stoppwort-Filterung, Stemming*) und speichert ihn in einem hoch-optimierten **Inverted Index**.

2. Erweiterte Suchfunktionen

Du kannst deinen gespeicherten Quellenschatz flexibel durchsuchen:

- **Exakte Phrasensuche:** Z. B. "künstliche Intelligenz in der Medizin"
- **Fuzzy Search (Tippfehlertoleranz):** Findet Begriffe auch bei Schreibfehlern.
- **Wildcards & Boole'sche Operatoren:** Verknüpfung von Suchbegriffen mit AND, OR, NOT.

3. Metadaten & Facetten-Filterung

Du kannst jedem gespeicherten Dokument zusätzliche Felder zuweisen:

- **Titel, URL, Speicherdatum, Autor, Schlagworte**
- Filtere deine Recherche gezielt nach Datumsbereichen oder Themenkategorien.

4. KI-basierte Vektorsuche (Ähnlichkeitssuche)

Moderne Lucene-Versionen unterstützen **Dense Vector Search**:

- Generiere Embeddings (KI-Vektoren) aus den Textabschnitten deiner Webseiten.
- Finde Absätze, die *bedeutungsmäßig* ähnlich zu deiner Fragestellung sind – selbst wenn nicht dieselben Wörter verwendet werden.

Praxis-Tipp

Wenn du nicht die gesamte Infrastruktur in Java selbst schreiben möchtest, nutze **Elasticsearch** oder **OpenSearch**. Beide bauen direkt auf Lucene auf, bieten aber sofort einsatzbereite REST-APIs, ein Web-Interface (Kibana/OpenSearch Dashboards) und fertige Web-Crawler.

User prompt: Aber kann ich mehrere äh Indizes aufmachen? Angenommen, ich habe verschiedene Projekte und ich recherchiere in jedem Projekt einzeln. Kann ich dann für jedes Projekt einen eigenen Index aufmachen?

Response: **Ja, das geht problemlos.** Du kannst für jedes Projekt einen eigenen Index anlegen, indem du einfach für jedes Projekt einen separaten Verzeichnispfad auf der Festplatte angibst.

Möglichkeiten zur Trennung:

- **Eigene Ordner (Eigene Indizes):** Du erstellst pro Projekt einen eigenen IndexWriter mit einem eigenen Ordnerpfad (z. B. /indices/projekt_a/ und /indices/projekt_b/). Die Daten sind somit vollkommen isoliert.
- **Ein gemeinsamer Index mit Projekt-Feld:** Du nutzt nur *einen* Index und fügst jedem Dokument ein Metadaten-Feld projekt_id hinzu. Bei der Suche filterst du dann einfach nach projekt_id: "projekt_a".

Welcher Ansatz passt wann?

Ansatz	Vorteile
Separater Index pro Projekt	Perfekt für strikte Mülltrennung. Löscht oder archivierst du ein Projekt, löschst du einfach den Ordner.
Ein Index mit Filter-Feld	Besser, wenn du projektübergreifend suchen willst oder sehr viele kleine Projekte hast.

Suchst du nach einem Code-Beispiel in Java für die Verwaltung mehrerer Indizes?

User prompt: Ja, also ich würde das im Grunde wie so ein IntelliJ Projekt aufziehen, würde ich sagen, da könnte man ja noch so weiß nicht, Verwaltungsdaten mit in das Projekt speichern. Also ich würde auf jeden Fall einen eigenen Ordner anlegen.

Response: Das ist eine sehr solide Architektur. Wenn du jedes Recherche-Projekt wie ein eigenständiges Ordner-Paket behandelst, bleibt deine Anwendung modular und leicht zu sichern.

Mögliche Ordnerstruktur:

```
mein-recherche-projekt/  
├── .index/           <-- Lucene speichert hier seine Index-Segmente  
├── metadata.json    <-- Projektname, Erstellungsdatum, Notizen  
└── raw_files/       <-- Gespeicherte HTML-/PDF-Dateien
```

Code-Beispiel (Java):

```
import org.apache.lucene.analysis.standard.StandardAnalyzer;  
import org.apache.lucene.index.IndexWriter;  
import org.apache.lucene.index.IndexWriterConfig;  
import org.apache.lucene.store.Directory;  
import org.apache.lucene.store.FSDirectory;  
  
import java.io.IOException;  
import java.nio.file.Paths;  
  
public class ProjectIndexManager {  
  
    public IndexWriter createProjectIndex(String projectFolderPath) throws IOException {  
        // Pfad zum Unterordner ".index" im Projektordner  
        Directory dir = FSDirectory.open(Paths.get(projectFolderPath, ".index"));  
  
        StandardAnalyzer analyzer = new StandardAnalyzer();  

```

```

IndexWriterConfig config = new IndexWriterConfig(analyzer);

// Erstellt den IndexWriter speziell für dieses Projekt
return new IndexWriter(dir, config);
}
}

```

Willst du auch ein Beispiel dazu sehen, wie du HTML-Webseiten vor der Indexierung am besten in sauberen Text umwandelst (z. B. mit JSoup)?

User prompt: Also dieses Projekt soll im Endeffekt eine wissenschaftliche Arbeit werden. Also jedes Projekt ist wieder eine wissenschaftliche Arbeit für sich und eine wissenschaftliche Arbeit noch eigentlich so, dass man erstmal das Inhaltsverzeichnis schreibt, dann hat man außerdem auch noch ein Konzeptpapier. Ähm, ich weiß nicht, was noch alles, aber das muss auf jeden Fall irgendwie in diesem Projekt abgebildet werden.

Response: Um ein solches Forschungsprojekt vollständig abzubilden, kannst du die Ordnerstruktur erweitern und sowohl strukturierte Dokumente (Konzept, Gliederung) als auch die Such-Engine miteinander verzahnen.

Erweiterte Projektstruktur

```

mein-wissenschaftliches-projekt/
├── .index/                <-- Lucene-Volltextindex (für schnelle Quellensuche)
├── project-info.json      <-- Metadaten (Titel, Deadline, Status)
├── concept.md             <-- Dein Konzeptpapier / Exposee
├── outline.json           <-- Inhaltsverzeichnis / Kapitelstruktur
├── sources/              <-- Eigene Quellensammlung
│   ├── raw_web/          <-- Gespeicherte HTML-Seiten
│   └── pdfs/             <-- Fachliteratur (PDFs)
└── notes/                <-- Eigene Notizen zu einzelnen Kapiteln

```

Wie du die Elemente sinnvoll verbindest

- **Kapitelbasierte Indexierung:** Du kannst jedem recherchierten Dokument im Lucene-Index nicht nur den Text mitgeben, sondern auch ein Feld `chapter_id` (z. B. `1.2_grundlagen`). So findest du sofort alle recherchierten Quellen, die du einem spezifischen Kapitel deines Inhaltsverzeichnisses zugeordnet hast.
- **Dynamische Gliederung (outline.json):** Speichere das Inhaltsverzeichnis strukturiert ab, damit deine Java-Anwendung dir zu jedem Kapitel direkt die passenden Quellen, Status-Updates und Notizen anzeigen kann:

```

{
  "title": "KI in der Medizin",
  "chapters": [
    { "id": "1.1", "title": "Einleitung", "status": "DONE" },
    { "id": "2.1", "title": "Verwandte Arbeiten", "status": "IN_PROGRESS" }
  ]
}

```

Möchtest du sehen, wie man in Lucene ein Suchfeld für bestimmte Kapitel-Zuordnungen anlegt?

User prompt: Ich will erstmal gar nichts implementieren, ich will aktuell nur die Anforderung definieren, wie das überhaupt aussehen soll.

Response: Das macht absolut Sinn. Um die Anforderungen sauber zu definieren, solltest du dein System in drei Kernbereiche unterteilen:

1. Daten- & Projektstruktur

- **Projekt-Container:** Jedes Recherche-Projekt ist ein abgekapselter Ordner (inkl. eigenen Lucene-Index).
- **Dokumententypen:** Unterscheidung zwischen Arbeitsdokumenten (*Exposé/Konzept, Inhaltsverzeichnis/Gliederung, Notizen*) und Quellenmaterial (*Webseiten-Dumps, PDFs*).

- **Metadaten:** Zuordnung von Quellen zu bestimmten Kapiteln, Tags oder Bearbeitungszuständen (z. B. "gelesen", "zitiert").

2. Funktionalitäten & Workflows

- **Import & Erfassung:** Speichern von Webseiten (HTML-Text extrahieren) und PDFs per Befehl oder Drag-and-Drop.
- **Gliederungs-Verwaltung:** Erstellen und Anpassen eines dynamischen Inhaltsverzeichnisses.
- **Suchen & Filtern:** * Volltextsuche über alle Quellen eines Projekts.
 - Gezielte Suche nach Kapiteln ("Zeige mir alle Quellen für Kapitel 2.1").
 - Semantische Ähnlichkeitssuche (KI/Vektoren), um passende Textstellen zu Konzept-Notizen zu finden.

3. Schnittstellen & Export

- **Zitat- & Quellenausgabe:** Export von Verweisen und Zitaten für die spätere Arbeit.
- **Projekt-Export:** Sichern oder Verschieben des gesamten Projekt-Ordners als eigenständiges Archiv.

Sollen wir als Nächstes die genauen Spezifikationen für die **Gliederung** oder für das **Quellen-Management** ausarbeiten?

User prompt: Ja, mach das.

Response: Hier ist der detaillierte Anforderungskatalog für die beiden Kernbereiche:

1. Anforderungen: Gliederung & Konzept

- **Hierarchische Struktur:**
 - Erstellen, Bearbeiten und Verschieben von Kapiteln (z. B. 1.1, 1.2, 2.1).
- **Verknüpfung mit Konzept-Notizen:**
 - Jedes Kapitel kann eine eigene Notiz enthalten (z. B. Forschungsziel, Kernargumentation).
- **Status-Tracking:**
 - Kapitel-Status verwalten (z. B. Geplant, In Recherche, Entwurf, Fertig).
- **Verlinkung zu Quellen:**
 - M:N-Beziehung zwischen Kapiteln und gesammelten Quellen (eine Quelle kann mehreren Kapiteln zugewiesen werden).

2. Anforderungen: Quellen-Management

- **Import & Aufbereitung:**
 - Speichern von Webseiten-Inhalten und Dokumenten (z. B. PDF, HTML, TXT).
 - Automatisches Extrahieren von Reintext und Entfernen von Stördaten (z. B. Menüleisten auf Webseiten).
- **Metadaten-Erfassung:**
 - Manuelle und automatische Erfassung von: *Titel, URL, Autor, Speicherdatum, Publikationsdatum, Notizen*.
- **Status der Quelle:**
 - Zuordnung von Bearbeitungszuständen (z. B. Ungelesen, Wichtig, Zitiert, Verworfen).
- **Gezielter Volltext-Index:**
 - Der extrahierte Text landet im projekt-eigenen Lucene-Index.
 - Filtermöglichkeiten nach Kapitel-ID, Status, Tag oder Datumsbereich.

Sollen wir als Nächstes die Anforderungen für die **Suchfunktionen** genauer definieren oder die **Benutzeroberfläche (UI)** betrachten?

User prompt: Also die Oberfläche ist eigentlich nur sekundär, das ist nur damit der Nutzer vielleicht ein bisschen was korrigieren kann, vielleicht eine Quelle verschieben kann in ein bestimmtes Kapitel und solche Sachen, aber eigentlich soll dieses Tool von einer KI bedient werden mittels MCP Tools.

Response: Das verändert die Architektur entscheidend: Die Hauptschnittstelle ist kein komplexes Frontend, sondern eine **MCP-Server-Schnittstelle (Model Context Protocol)**, über die ein AI-Agent das Projekt autonom steuert.

1. Anforderungen: MCP-Tools (KI-Schnittstelle)

Die KI benötigt spezifische Funktionen (Tools), um das Projekt zu lesen und zu strukturieren:

- **Projekt & Gliederung:**
 - `get_outline()`: Liest die aktuelle Kapitelstruktur und Status aus.
 - `update_outline(json_structure)`: Erstellt oder verändert Kapitel und Abschnitte.
 - `set_chapter_concept(chapter_id, text)`: Hinterlegt die Argumentation/Notiz für ein Kapitel.
- **Recherche & Indexierung:**
 - `ingest_source(url_or_file, chapter_ids[])`: Lädt Webseiten/PDFs, wandelt sie in Text um, indexiert sie in Lucene und weist sie Kapiteln zu.
 - `search_sources(query, filters)`: Durchsucht den Volltext-Index gezielt nach Begriffen, Tags oder Kapiteln.
 - `find_similar_content(text_snippet)`: Nutzt Vektorsuche, um passende Quellen für einen Entwurf zu finden.
- **Status & Reporting:**
 - `get_project_summary()`: Liefert den aktuellen Fortschritt, offene Recherche-Lücken und zugewiesene Quellen.

2. Anforderungen: Minimale Nutzeroberfläche (Human-in-the-Loop)

Die UI dient primär der Kontrolle und Korrektur von KI-Aktionen:

- **Visualisierung:** Übersicht über das Inhaltsverzeichnis mit zugeordneten Quellen und Statusanzeige.
 - **Manuelle Anpassungen:** Drag-and-Drop zum Verschieben von Quellen zwischen Kapiteln.
 - **Metadaten-Korrektur:** Manuelles Bearbeiten von Quellangaben (Titel, Autor, Jahr), falls die automatische Extrahierung fehlerhaft war.
 - **Inspektion:** Einsicht in den Extraktions-Text und die Notizen der KI.
-

User prompt: Die Tools müssen außer extrem simpel sein, weil wir wollen das mit einer KI mit einer einfachen KI z.B. GPT OSS 20B machen, also die braucht natürlich massive Toolunterstützung, aber das müssen einfache Tools sein und wir müssen das immer alles sehr häppchenweise bearbeiten, das ist halt ganz wichtig.

Response: Damit ein kleineres Modell wie GPT OSS 20B nicht überfordert wird, muss jedes Tool **genau eine atomare Aufgabe** erfüllen und **minimale Payloads** liefern. Die KI arbeitet sich so Schritt für Schritt durch das Projekt.

1. Atomare MCP-Tools (Anforderungen)

Gliederung & Struktur (Sehr klein geschnitten)

- `get_outline_tree()`: Liefert *nur* die IDs und Titel aller Kapitel (keinen Volltext).
- `add_chapter(parent_id, title)`: Erstellt ein neues Kapitel.

- `get_chapter_details(chapter_id)`: Holst Notizen, Status und Quell-IDs *nur* für dieses eine Kapitel.
- `set_chapter_status(chapter_id, status)`: Ändert lediglich den Status (TODO, DONE).

Quellen & Suche (Häppchenweise)

- `ingest_url(url)`: Lädt Seite, speichert sie in Lucene und gibt *nur* eine `source_id` zurück.
- `search_sources(query, limit)`: Sucht im Index und liefert maximal 3–5 kurze Text-Snippets inkl. `source_id`.
- `get_source_text(source_id, offset, length)`: Liest Text einer Quelle in kleinen Abschnitten (Paging), um das Kontextfenster nicht zu sprengen.
- `link_source_to_chapter(source_id, chapter_id)`: Verknüpft eine Quelle mit einem Kapitel.

2. Der "Häppchen-Workflow" (Beispiel)

Anstatt "Recherchiere Kapitel 2" läuft der Prozess in kleinen Schleifen ab:

1. KI ruft `get_outline_tree()` auf → wählt `kapitel_2`.
2. KI ruft `search_sources("Suchbegriff", limit=3)` auf → erhält 3 kurze Snippets.
3. KI entscheidet für Snippet 1 und ruft `link_source_to_chapter("source_123", "kapitel_2")` auf.
4. KI ruft `set_chapter_status("kapitel_2", "RESEARCHED")` auf.

Durch diese extrem kurze Tool-Chain bleibt das Prompt-Token-Limit klein und die Fehlerquote des 20B-Modells minimal.

User prompt: Also, ich will eigentlich die KI auch zu ähm bewusst zu diesen wissenschaftlichen Arbeiten zwingen. Also, ich würde das so machen wollen, dass der Benutzer sagt, wir arbeiten jetzt erstmal das Inhaltsverzeichnis aus und das wird auch von der UI quasi so vorgegeben, dass das zuerst gemacht werden soll. Und äh ohne Inhaltsverzeichnis äh verarbeit äh verweigert das Programm einfach seinen Dienst und sagt, äh mach erstmal das Inhaltsverzeichnis. Ähm und dabei kann aber die KI auch unterstützen und die kann das vielleicht auch dann kapitelweise machen. Ähm wichtig wäre halt auch irgendwie vielleicht noch das Konzeptpapier, dass man das auch vielleicht noch mal vorher erstellt, dass man eine grobe Ahnung hat, was man überhaupt vorhat. Ähm das kann man vielleicht auch verfeinern und dann dementsprechend dann die Kapitel noch mal anpassen. Ähm und müsste das wahrscheinlich auch in mehreren Schleifen dann immer machen. Also, wir müssen irgendwie das Maximum aus einer einfachen KI rausholen.

Response: Um ein kleineres Modell (wie GPT OSS 20B) zu Höchstleistungen zu treiben, nutzt du am besten eine **strenge State Machine (Zustandssteuerung)** im Backend. Das System schaltet Funktionen erst dann frei, wenn der vorherige Schritt abgeschlossen ist.

1. Der phasenbasierte Workflow (State Machine)

Das System erzwingt eine feste Reihenfolge. Die KI bekommt pro Phase nur die dafür zugelassenen Tools bereitgestellt:

[Phase 1: Konzept] → [Phase 2: Gliederung] → [Phase 3: Kapitelweise Recherche]

1. Phase 1: Konzeptpapier

- **Sperre:** Es können noch keine Quellen gesucht oder Kapitel angelegt werden.
- **Ziel:** Die KI hilft dem Nutzer, Fragestellung, Ziel und Methodik in 2–3 Iterationen zu schärfen.
- **Tool:** `save_concept_draft(text)`

2. Phase 2: Inhaltsverzeichnis (Gliederung)

- **Sperre:** Das System verweigert Recherche-Tools, bis eine valide Struktur vorliegt.
- **Ziel:** Auf Basis des Konzeptpapiers erstellt die KI schrittweise (häppchenweise) Kapitel für Kapitel.
- **Tools:** `add_chapter(title, parent_id)`, `finalize_outline()`

3. Phase 3: Iterative Recherche & Schreiben

- **Freischaltung:** Die Volltextsuche und Quellen-Zuordnung werden aktiviert.
- **Ziel:** Die KI arbeitet sich **strikte Kapitel für Kapitel** durch die Arbeit (z. B. Konzept ansehen → Suchbegriffe generieren → Quellen suchen → Notizen zuordnen).

2. Wie du das Maximum aus der 20B-KI herausholst

- **Gezielte System-Prompts je Phase:** Das Backend schickt je nach Phase einen anderen System-Prompt mit. In Phase 1 weiß die KI nicht einmal, dass es spätere Recherche-Tools gibt. Das spart Tokens und verhindert Halluzinationen.
- **Refinement-Schleifen:** Nach jedem Schritt (z. B. Erstellung von Kapitel 1) fragt das System die KI: *"Passt diese Gliederung noch zum Konzeptpapier? Antworte mit JA oder NEIN."* Bei NEIN wird automatisch eine Korrektur-Schleife angestoßen.
- **UI als "Türsteher":** Die Benutzeroberfläche schaltet die Buttons für den nächsten Schritt erst frei, wenn der aktuelle Zustand vom Backend als COMPLETED markiert wurde.

User prompt: Bist du dir sicher, dass man tatsächlich schon ein Inhaltsverzeichnis umschreiben kann, ohne recherchiert zu haben? Ich würde fast eher sagen, es kommt zwar erst das Konzeptpapier, aber dann würde man erstmal recherchieren mit dem Konzept, was man hat und dann würde man das Inhaltsverzeichnis schreiben und dann würde man vielleicht noch mal detaillierter recherchieren.

Response: Da hast du völlig recht – in der Praxis läuft das tatsächlich meistens genau so ab. Ein realistischer wissenschaftlicher Workflow braucht zwei Recherche-Phasen: Erst eine **explorative Suche** für den Überblick und danach eine **zielgerichtete Suche** für die einzelnen Abschnitte.

So sieht die angepasste Pipeline für dein System aus:

Der angepasste Workflow (State Machine)

[1. Konzept] → [2. Orientierungs-Recherche] → [3. Gliederung] → [4. Detail-Recherche & Entwurf]

1. Phase 1: Konzeptpapier

- Formulierung von Problemstellung, Ziel und Leitfragen.

2. Phase 2: Orientierungs-Recherche (Grob-Recherche)

- **Ziel:** Das Thema überblicken und den Stand der Forschung erfassen.
- **Tools:** Breit angelegte Suchen im Lucene-Index, um relevante Kernbegriffe, Standardwerke und Themencuster zu identifizieren.

3. Phase 3: Inhaltsverzeichnis (Gliederung)

- **Ziel:** Struktur aufbauen.
- **Logik:** Die KI nutzt das Konzept *zusammen* mit den Funden aus der Orientierungs-Recherche, um ein fundiertes Inhaltsverzeichnis zu generieren.

4. Phase 4: Kapitelweise Detail-Recherche

- **Ziel:** Strenge, fokussierte Quellensuche pro Kapitel-ID für das tatsächliche Schreiben.

Sollen wir für Phase 2 (Orientierungs-Recherche) festlegen, wie die KI die Suchergebnisse zusammenfassen soll, bevor sie daraus die Gliederung baut?

User prompt: Nein, wir suchen nicht im Lucin Index, der ist der ist nachher für die Schreibarbeit gedacht. Ich will im Internet suchen, ich will das automatisch die KI per Browserautomatisierung anhand des Konzepts sich selbst einen Überblick verschafft, die Sachen im Lucin Index speichert, damit sie die nachher wieder nachschlagen kann, wenn sie das Inhaltsverzeichnis schreibt und eben effektiv drauf zugreifen kann und dann wenn das Inhaltsverzeichnis steht, dann könnte sie für die Kapitel jeweils wieder recherchieren.

Response: Das ist ein extrem starker Ansatz für eine Automatisierungs-Pipeline.

Der angepasste KI-Workflow

[1. Konzept] → [2. KI-Web-Crawl & Lucene-Ablage] → [3. Gliederung] → [4. Detaillierte Kapitel-Recherche]

1. **Phase 1: Konzept festlegen** Das Konzept dient als Basis für den Recherche-Auftrag der KI.
2. **Phase 2: Automatische Orientierungs-Recherche (Browser) * Suchauftrag:** Die KI generiert aus dem Konzept breite Suchbegriffe und steuert via Browser-Automatisierung (z. B. Playwright oder Puppeteer) das Web an.
 - **Befüllen:** Sie zieht den Inhalt der gefundenen Seiten ab, bereinigt den Text und speichert ihn direkt im **Lucene-Index**.
3. **Phase 3: Gliederung erstellen** Die KI liest gezielt in ihren frisch indexierten Lucene-Daten nach und baut daraus zusammen mit dem Konzept das Inhaltsverzeichnis auf.
4. **Phase 4: Zielgerichtete Kapitel-Recherche** Für gezielte Wissenslücken in den einzelnen Kapiteln stößt die KI erneut punktuelle Web-Suchen an und erweitert den Index.

Benötigte MCP-Tools für Phase 2

- `search_web(query)`: Nutzt die Browser-Automatisierung und liefert eine Liste passender URLs.
- `scrape_and_index_url(url)`: Ruft die Webseite auf, zieht den Text ab, simuliert gegebenenfalls Klicks und speichert den Inhalt direkt in den Lucene-Index.

Möchtest du als Nächstes festlegen, wie die KI feststellt, wann sie *genug* orientierend recherchiert hat, um mit der Gliederung zu beginnen?

User prompt: Ja, mach das.

Response: Das Abbruchkriterium lässt sich über zwei einfache Mechanismen steuern:

1. **Maximales Budget (Hard Limit):** Das System erlaubt fest **X Webseiten** oder **Y Suchanfragen** pro Phase. So läuft der Browser nie endlos.
2. **Eigenbewertung der KI:** Nach jedem Crawl-Durchlauf prüft die KI anhand von 3 Fragen, ob das Thema ausreichend abgedeckt ist:
 - *Kenne ich die 3–5 zentralen Fragestellungen des Themas?*
 - *Habe ich die wichtigsten Fachbegriffe erfasst?*
 - *Kann ich die Hauptperspektiven/Ansätze benennen?*

Sobald die KI dreimal hintereinander **JA** antwortet oder das Budget verbraucht ist, schaltet das Backend automatisch in die nächste Phase (Gliederung) um.

User prompt: Also, ich würde sagen, dass wir uns da vielleicht auch wieder mehr auf den Human in the Loop verlassen, weil ich würde das einer einfachen KI nicht alleine überlassen, sondern die Steuerung sollte dann noch der Mensch übernehmen.

Response: Das ist ein sehr schlauer Kniff. Das spart der KI komplexe Logik und gibt dem Nutzer die volle Kontrolle zurück.

Der Human-in-the-Loop-Workflow

- **Orientierungs-Recherche:** Die KI schlägt z. B. 3–5 Webseiten vor, crawlt diese auf Befehl und legt sie in Lucene ab.
- **Freigabe-Schleife:** Die UI zeigt dem Nutzer die gesammelten Quellen und einen Button "**Genug recherchiert – Gliederung starten**".
- **Mensch entscheidet:** Erst wenn der Nutzer klickt, wechselt das Backend die Phase und schaltet die Gliederungs-Tools frei.

Damit dient die KI rein als fleißiger Assistent, während der Mensch die Regie behält.

User prompt: Ähm, ich weiß nicht, wie das bei einfachen Tools ist und wie das mit Olama ist, aber es wäre natürlich cool, wenn er gleichzeitig denken könnte, wenn er gleichzeitig also sagen würde, was er tut, wonach er recherchiert und vielleicht auch dann zusammenfasst, was seine Erkenntnisse sind und dann eben darauf aufbauen weitersucht, so dass sich da quasi so ein Fluss ergibt, was der User quasi mitlesen kann und dann kann er vielleicht zu der Erkenntnis kommen, wann genug Informationen zusammen sind und wann das aggregiert ist oder er kann vielleicht auch die KI unterbrechen und sagen, fast jetzt mal zusammen, was du bisher hast und dann entscheidet der User, okay, das reicht oder dann sagt er, nee, recherchier weiter. Also halb manuell, halbautomatisch das Ganze.

Response: Das trifft das Prinzip des **Streamings von Tool-Calls und CoT (Chain of Thought)** exakt. Bei lokalen Modellen wie in Ollama lässt sich der Gedankengang ("Reasoning") der KI live an die UI streamen, sodass der Nutzer in Echtzeit mitliest.

So sieht die Schnittstelle für diesen Fluss aus:

1. Live-Protokollierung (Streaming):

- **Denkschritt:** "Ich erstelle Suchbegriffe basierend auf Konzept-Punkt 2..."
- **Aktion:** `search_web("Suchbegriff")`
- **Erkenntnis:** "Die Quelle beschreibt Ansatz X, aber Y fehlt noch. Ich suche weiter nach Y."

2. Interaktive Steuerung durch den Nutzer:

- **Mitlesen:** Der Nutzer sieht den Recherche-Stream live in einem Dashboard.
- **Unterbrechen-Befehl:** Ein Button in der UI sendet ein PAUSE-Signal an den Backend-Workflow.
- **Zusammenfassung anfordern:** Das Backend ruft das Tool `summarize_current_index()` auf, und die KI gibt einen Zwischenstand aus.
- **Entscheidung:** Der Nutzer klickt entweder auf "**Weiter recherchieren**" oder schaltet das System manuell in die Phase "**Gliederung erstellen**".

Dieser Ansatz kombiniert die hohe Autonomie der KI mit vollkommener Transparenz und der vollen Kontrolle für den Nutzer.

User prompt: Genauso wollen wir das machen. Kannst du das Konzept ausarbeiten, die Anforderung schreiben dafür?

Response: Here is the system requirements specification for the interactive, AI-assisted research pipeline:

1. Systemarchitektur & Ablaufsteuerung

- **State Machine:** Ein Backend-State-Manager erzwingt eine feste Phasenfolge:
 1. **Phase 1: Konzeptentwicklung** (Problemstellung, Ziel, Leitfragen)
 2. **Phase 2: Explorative Orientierungs-Recherche** (Live-Web-Crawl, Lucene-Ablage)
 3. **Phase 3: Gliederung & Inhaltsverzeichnis** (Kapitelstruktur)
 4. **Phase 4: Kapitelweise Detail-Recherche & Entwurf**
- **Strict Phase Gating:** Das Backend gibt für das Ollama-Modell je nach Phase nur spezifische MCP-Tools frei. Phasenwechsel erfolgen erst nach expliziter Nutzerfreigabe in der UI.

2. Interaktives Live-Streaming (Human-in-the-Loop)

- **Thought- & Action-Streaming:** Die Gedanken der KI (*Reasoning*), geplante Tool-Aufrufe und gefundene Erkenntnisse werden über WebSockets in Echtzeit an die UI gestreamt.
- **Manuelle Intervention:** Der Nutzer kann den laufenden Recherche-Loop jederzeit per UI-Action unterbrechen.
- **Zwischenauswertung:** Nach einer Unterbrechung ruft das System ein Zusammenfassungs-Tool auf. Der Nutzer entscheidet anschließend:
 - **Option A:** Weiter recherchieren (mit optionalen Zusatzanweisungen).

- **Option B:** Phase abschließen und in die Gliederung wechseln.

3. Spezifikation der MCP-Tools

Phase 1: Konzept

- `save_concept(draft_text)`: Speichert die aktuelle Fassung des Konzeptpapiers.
- `get_concept()`: Liest das aktuell festgelegte Konzept aus.

Phase 2 & 4: Browser-Automatisierung & Lucene-Index

- `search_web(query, limit)`: Führt eine Websuche per Browser-Automatisierung durch und liefert URLs/Snippets.
- `scrape_and_index_url(url)`: Extrahiert Reintext aus Webseiten und speichert diesen inklusive Metadaten (URL, Zeit) im projekt-eigenen Lucene-Index.
- `summarize_index_content()`: Aggregiert den bisherigen Wissensstand aus dem Lucene-Index als Zwischenstand für den Nutzer.

Phase 3: Gliederung

- `get_index_overview()`: Greift auf den Lucene-Index zu, um Themencluster für die Gliederung zu analysieren.
- `create_outline(json_structure)`: Speichert das strukturierte Inhaltsverzeichnis (Kapitel-IDs und Titel).
- `add_chapter_note(chapter_id, note)`: Verknüpft spezifische Vorgaben oder Konzeptteile mit einzelnen Kapiteln.

4. Anforderungen an die UI (Steuer-Dashboard)

- **Live-Feed:** Ein Konsolen-Fenster zeigt den Denkprozess und die Tool-Aktionen der KI chronologisch an.
 - **Steuerungselemente:**
 - Button: "**Recherche pausieren & Zwischenstand anzeigen**"
 - Button: "**Recherche fortsetzen**"
 - Button: "**Orientierungs-Recherche beenden → Gliederung starten**"
 - **Inspektion:** Ein Bereich zur Ansicht der indexierten Quellen und zur manuellen Feinanpassung des Inhaltsverzeichnisses.
-